

Phase 4: Selection Sort — Full Structured Beginner Note

1. Current Progress Before Phase 4

Before learning Selection Sort, you have already studied three important phases.

Phase 1: Sorting Foundations

In Phase 1, you learned the basic language used to understand sorting algorithms.

You learned words like:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already or nearly sorted
Non-adaptive	Does not become faster on already sorted data
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(1)$	Constant time or constant space
$O(n)$	Linear time
$O(n^2)$	Quadratic time
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, "Does it sort?"

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

In Phase 2, you learned Bubble Sort.

Bubble Sort works by comparing neighboring elements.

Example:

8 5

Since $8 > 5$, Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
 - It swaps values when they are in the wrong order.
 - One full scan is called a pass.
 - After each pass, one large value is fixed at the end.
 - After k passes, the k largest values are fixed at the end.
 - It can be adaptive if we use a `flag`.
 - It is stable if we use `>` instead of `>=`.
 - Best-case time is $O(n)$ when the adaptive flag is used.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
-

Phase 3: Insertion Sort

In Phase 3, you learned Insertion Sort.

Insertion Sort works by taking one value and inserting it into the correct place inside a sorted left part.

Example:

[5 8] 7 3 2

The left part [5 8] is sorted.

Now we insert 7 into the correct position:

[5 7 8] 3 2

Important points from Insertion Sort:

- It builds a sorted left part.
- It starts from index 1 because the first element alone is already sorted.
- It stores the selected value in `x`.
- It shifts bigger values to the right.
- It inserts `x` at the correct position.
- It is naturally adaptive.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- It is good for nearly sorted data.

2. Why Phase 4 Comes After Insertion Sort

Bubble Sort, Insertion Sort, and Selection Sort are all simple comparison-based sorting algorithms.

They are usually taught together because all three are easy to understand and all three can have $O(n^2)$ time complexity.

But they move data in different ways.

Algorithm	Main idea	Movement style
Bubble Sort	Push large values to the end	Many neighboring swaps
Insertion Sort	Insert each value into the sorted left part	Shifting
Selection Sort	Select the smallest value and place it at the front	Few swaps

Selection Sort is important because it teaches a new strategy:

Instead of swapping neighbors or inserting values, we select the smallest value and put it in the correct position.

This makes Selection Sort very different from Bubble Sort and Insertion Sort.

3. Main Goal of Phase 4

The main goal of Phase 4 is to understand Selection Sort.

Selection Sort teaches these important ideas:

- Selecting the correct value for a position.
 - Finding the minimum value from the unsorted part.
 - Using `i`, `j`, and `k` indexes.
 - Swapping only after scanning the unsorted part.
 - Understanding why Selection Sort uses fewer swaps.
 - Understanding why Selection Sort is not adaptive.
 - Understanding why Selection Sort is not stable.
 - Understanding the `k`-pass result.
 - Writing Selection Sort in C.
 - Comparing Selection Sort with Bubble Sort and Insertion Sort.
-

4. What Is Selection Sort?

Selection Sort is a sorting algorithm that repeatedly selects the smallest value from the unsorted part of the array and places it at the beginning of that unsorted part.

In simple words:

Select the smallest value and put it where it belongs.

Selection Sort works from left to right.

It fills the array position by position.

At each pass, it asks:

Which value should go in this position?

Then it searches the remaining array, finds the smallest value, and swaps it into that position.

5. Real-Life Analogy: Choosing the Smallest Item

Imagine you have several books with different heights.

You want to arrange them from shortest to tallest.

You do this:

1. Look at all the books.
2. Find the shortest book.
3. Put it in the first position.
4. Look at the remaining books.
5. Find the next shortest book.
6. Put it in the second position.
7. Repeat until all books are arranged.

That is Selection Sort.

It keeps selecting the smallest remaining item.

6. Main Idea of Selection Sort

Selection Sort works like this:

1. Start at the first position.
2. Assume the first value is the smallest.
3. Scan the rest of the array.
4. If you find a smaller value, remember its index.
5. After the scan finishes, swap the smallest value with the current position.
6. Move to the next position.
7. Repeat until the array is sorted.

Short memory trick:

Select minimum, swap once.

7. The Two Parts of the Array

During Selection Sort, the array can be thought of as two parts:

sorted part | unsorted part

At the beginning, nothing is sorted yet:

| 8 6 3 2 5 4

After one pass, the smallest value is fixed at the front:

```
2 | 6 3 8 5 4
```

After two passes:

```
2 3 | 6 8 5 4
```

After three passes:

```
2 3 4 | 8 5 6
```

The sorted part grows from left to right.

8. Important Difference from Bubble Sort and Insertion Sort

Bubble Sort

Bubble Sort pushes large values to the end.

After each pass:

```
One largest value is fixed at the end.
```

Insertion Sort

Insertion Sort builds a sorted left part.

After each pass:

```
The left part is sorted, but values may not be in final global positions yet.
```

Selection Sort

Selection Sort selects the smallest value and places it at the front.

After each pass:

One smallest value is fixed at the front.

This is the key idea of Phase 4.

9. The Three Important Variables: `i`, `j`, and `k`

Selection Sort usually uses three index variables:

Variable	Meaning
<code>i</code>	The position we are trying to fill
<code>j</code>	The scanner that searches the remaining array
<code>k</code>	The index of the smallest value found so far

Think of them like this:

`i` = where I want to place the next smallest value
`j` = the person searching through the array
`k` = the best/smallest value found so far

10. Meaning of `i`

The variable `i` chooses the current position.

Example:

8 6 3 2 5 4

When `i = 0`, Selection Sort is trying to find the correct value for index `0`.

When `i = 1`, Selection Sort is trying to find the correct value for index `1`.

When `i = 2`, Selection Sort is trying to find the correct value for index `2`.

So `i` means:

This is the position I am filling now.

11. Meaning of `j`

The variable `j` scans the unsorted part of the array.

If `i = 0`, then `j` scans from index `0` to the end.

If `i = 1`, then `j` scans from index `1` to the end.

If `i = 2`, then `j` scans from index `2` to the end.

So `j` means:

I am searching through the remaining values.

12. Meaning of `k`

The variable `k` stores the index of the smallest value found so far.

At the start of each pass:

```
k = i;
```

This means:

For now, I assume the value at index `i` is the smallest.

Then `j` scans the remaining array.

If `j` finds a smaller value, `k` changes.

Example:

```
8 6 3 2 5 4
```

At first:

```
i = 0  
k = 0  
A[k] = 8
```

Now scan:

- 6 is smaller than 8, so k becomes index 1.
- 3 is smaller than 6, so k becomes index 2.
- 2 is smaller than 3, so k becomes index 3.
- 5 is not smaller than 2, so k stays 3.
- 4 is not smaller than 2, so k stays 3.

At the end:

```
k = 3  
A[k] = 2
```

So the smallest value is at index 3.

13. Simple Example of Selection Sort

Suppose the array is:

```
8 6 3 2 5 4
```

We want ascending order:

```
2 3 4 5 6 8
```

Selection Sort starts at index 0.

It asks:

What is the smallest value from the whole array?

The smallest value is 2.

So Selection Sort swaps 8 and 2.

Result:

```
2 6 3 8 5 4
```

Now 2 is fixed.

It is in its final sorted position.

14. Full Dry Run of Selection Sort

Let us sort this array in ascending order:

8 6 3 2 5 4

Final answer should be:

2 3 4 5 6 8

15. Initial State

Original array:

8 6 3 2 5 4

At the beginning, no value is fixed yet.

| 8 6 3 2 5 4

The whole array is unsorted.

16. Pass 1: Find the Smallest Value for Index 0

Start:

8 6 3 2 5 4

Current position:

i = 0

Assume the first value is smallest:

```
k = 0  
A[k] = 8
```

Now scan the array using `j`.

Compare `8` and `6`.

`6` is smaller than `8`, so update `k`.

```
k = 1  
A[k] = 6
```

Compare `6` and `3`.

`3` is smaller than `6`, so update `k`.

```
k = 2  
A[k] = 3
```

Compare `3` and `2`.

`2` is smaller than `3`, so update `k`.

```
k = 3  
A[k] = 2
```

Compare `2` and `5`.

`5` is not smaller than `2`, so do not update `k`.

Compare `2` and `4`.

`4` is not smaller than `2`, so do not update `k`.

The smallest value is `2`.

Swap `A[i]` and `A[k]`.

Swap `8` and `2`.

Result:

2 6 3 8 5 4

Now 2 is fixed.

2 | 6 3 8 5 4

17. Pass 2: Find the Smallest Value for Index 1

Start:

2 | 6 3 8 5 4

The first value 2 is already fixed.

Now work on the unsorted part:

6 3 8 5 4

Current position:

i = 1

Assume A[1] is smallest:

k = 1
A[k] = 6

Scan the remaining array.

Compare 6 and 3.

3 is smaller than 6, so update k.

k = 2
A[k] = 3

Compare 3 and 8.

8 is not smaller than 3, so do not update k.

Compare 3 and 5.

5 is not smaller than 3, so do not update k.

Compare 3 and 4.

4 is not smaller than 3, so do not update k.

The smallest value in the unsorted part is 3.

Swap 6 and 3.

Result:

2 3 6 8 5 4

Now 2 and 3 are fixed.

2 3 | 6 8 5 4

18. Pass 3: Find the Smallest Value for Index 2

Start:

2 3 | 6 8 5 4

The sorted part is:

2 3

The unsorted part is:

6 8 5 4

Current position:

$i = 2$

Assume $A[2]$ is smallest:

$k = 2$
 $A[k] = 6$

Scan the remaining array.

Compare 6 and 8 .

8 is not smaller than 6 , so do not update k .

Compare 6 and 5 .

5 is smaller than 6 , so update k .

$k = 4$
 $A[k] = 5$

Compare 5 and 4 .

4 is smaller than 5 , so update k .

$k = 5$
 $A[k] = 4$

The smallest value in the unsorted part is 4 .

Swap 6 and 4 .

Result:

2 3 4 8 5 6

Now 2 , 3 , and 4 are fixed.

2 3 4 | 8 5 6

19. Pass 4: Find the Smallest Value for Index 3

Start:

2 3 4 | 8 5 6

The sorted part is:

2 3 4

The unsorted part is:

8 5 6

Current position:

$i = 3$

Assume $A[3]$ is smallest:

$k = 3$
 $A[k] = 8$

Scan the remaining array.

Compare 8 and 5.

5 is smaller than 8, so update k .

$k = 4$
 $A[k] = 5$

Compare 5 and 6.

6 is not smaller than 5, so do not update k .

The smallest value in the unsorted part is 5.

Swap 8 and 5.

Result:

2 3 4 5 8 6

Now 2, 3, 4, and 5 are fixed.

2 3 4 5 | 8 6

20. Pass 5: Find the Smallest Value for Index 4

Start:

2 3 4 5 | 8 6

The sorted part is:

2 3 4 5

The unsorted part is:

8 6

Current position:

i = 4

Assume A[4] is smallest:

k = 4
A[k] = 8

Compare 8 and 6.

6 is smaller than 8, so update k.

```
k = 5  
A[k] = 6
```

The smallest value in the unsorted part is 6.

Swap 8 and 6.

Result:

```
2 3 4 5 6 8
```

Final sorted array:

```
2 3 4 5 6 8
```

21. Summary of the Dry Run

Original array:

```
8 6 3 2 5 4
```

After pass 1:

```
2 6 3 8 5 4
```

After pass 2:

```
2 3 6 8 5 4
```

After pass 3:

```
2 3 4 8 5 6
```

After pass 4:

2 3 4 5 8 6

After pass 5:

2 3 4 5 6 8

Final sorted array:

2 3 4 5 6 8

22. Important Rule After Each Pass

In Selection Sort:

After k passes, the k smallest values are fixed at the front.

Example after 3 passes:

2 3 4 | 8 5 6

The first three values are already the three smallest values in sorted order.

This is different from Bubble Sort.

In Bubble Sort:

After k passes, the k largest values are fixed at the end.

In Selection Sort:

After k passes, the k smallest values are fixed at the front.

23. Why Selection Sort Needs $n - 1$ Passes

If an array has n elements, Selection Sort needs at most $n - 1$ passes.

Example:

```
8 6 3 2 5 4
```

There are 6 elements.

So:

```
n = 6  
n - 1 = 5
```

Selection Sort needs at most 5 passes.

Why not 6 passes?

Because after 5 elements are placed correctly, the remaining last element must already be in the correct position.

24. Selection Sort Pseudocode

Simple version:

```
repeat for each position:  
    find the smallest value in the unsorted part  
    swap it with the current position
```

Detailed version:

```
for i from 0 to n - 2:  
    k = i  
  
    for j from i to n - 1:  
        if A[j] < A[k]:  
            k = j  
  
    swap A[i] and A[k]
```

Explanation:

- **i** chooses the position to fill.

- `k = i` means we first assume the current position has the smallest value.
- `j` scans the remaining array.
- `A[j] < A[k]` checks whether a smaller value has been found.
- If a smaller value is found, `k` is updated.
- After the inner loop finishes, `A[i]` and `A[k]` are swapped.

25. Basic Selection Sort Code in C

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void SelectionSort(int A[], int n) {
    int i, j, k;

    for (i = 0; i < n - 1; i++) {
        k = i;

        for (j = i; j < n; j++) {
            if (A[j] < A[k]) {
                k = j;
            }
        }

        swap(&A[i], &A[k]);
    }
}

int main() {
    int A[] = {8, 6, 3, 2, 5, 4};
    int n = sizeof(A) / sizeof(A[0]);

    SelectionSort(A, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
}
```

```
    return 0;
}
```

Expected output:

Sorted array: 2 3 4 5 6 8

26. Explanation of the C Program

The program has three main parts:

1. `swap()` function
2. `SelectionSort()` function
3. `main()` function

27. The `swap()` Function

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

This function swaps two integer values.

It uses pointers because it must change the real values inside the array.

Meaning:

Code	Meaning
<code>int *x</code>	<code>x</code> is a pointer to an integer
<code>int *y</code>	<code>y</code> is a pointer to an integer
<code>*x</code>	Value stored at the address inside <code>x</code>
<code>*y</code>	Value stored at the address inside <code>y</code>
<code>temp</code>	Temporary variable used to protect one value

When Selection Sort calls `swap`, it calls it like this:

```
swap(&A[i], &A[k]);
```

The `&` means address of.

So this gives the addresses of the two array elements.

28. The `SelectionSort()` Function

```
void SelectionSort(int A[], int n)
```

This function receives:

Parameter	Meaning
<code>A[]</code>	The array to sort
<code>n</code>	Number of elements in the array

29. Important Variables in Selection Sort

Variable	Meaning
<code>i</code>	Current position being filled
<code>j</code>	Scanner that searches the unsorted part
<code>k</code>	Index of the smallest value found so far
<code>n</code>	Number of elements
<code>A[i]</code>	Value at the current position
<code>A[j]</code>	Value currently being scanned
<code>A[k]</code>	Smallest value found so far

30. Outer Loop Explanation

```
for (i = 0; i < n - 1; i++)
```

The outer loop controls the passes.

It chooses which position we are filling.

If there are 6 elements:

```
n = 6  
n - 1 = 5
```

So at most 5 passes are needed.

Examples:

- When `i = 0`, find the smallest value for position 0.
- When `i = 1`, find the smallest value for position 1.
- When `i = 2`, find the smallest value for position 2.

31. Resetting `k`

```
k = i;
```

This line is very important.

At the beginning of every pass, we reset `k` to `i`.

This means:

I assume the current position contains the smallest value.

Then the inner loop checks whether that assumption is true.

If we forget `k = i`, the algorithm may remember an old minimum index from the previous pass and give wrong results.

32. Inner Loop Explanation

```
for (j = i; j < n; j++)
```

The inner loop scans the unsorted part of the array.

It starts from `i` because values before `i` are already fixed.

Example:

2 3 | 6 8 5 4

If $i = 2$, the values before index 2 are already sorted.

So j only needs to scan:

6 8 5 4

33. Minimum Check Explanation

```
if (A[j] < A[k]) {  
    k = j;  
}
```

This means:

If the value at j is smaller than the smallest value found so far, update k .

So k always stores the index of the smallest value found so far.

Example:

8 6 3 2 5 4

At first:

```
k = 0  
A[k] = 8
```

When we find 6, update k .

When we find 3, update k .

When we find 2, update k .

At the end, k points to 2.

34. Swap After the Inner Loop

```
swap(&A[i], &A[k]);
```

This happens after the inner loop finishes.

This is one of the most important ideas in Selection Sort.

Selection Sort does not swap every time it finds a smaller value.

It only remembers the smallest value using `k`.

Then, after the full scan, it performs one swap.

That is why Selection Sort uses fewer swaps than Bubble Sort.

35. The `main()` Function

```
int main() {  
    int A[] = {8, 6, 3, 2, 5, 4};  
    int n = sizeof(A) / sizeof(A[0]);  
  
    SelectionSort(A, n);  
  
    printf("Sorted array: ");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", A[i]);  
    }  
  
    printf("\n");  
    return 0;  
}
```

This function:

1. Creates an array.
 2. Finds the number of elements.
 3. Calls `SelectionSort()`.
 4. Prints the sorted array.
-

36. Meaning of `sizeof(A) / sizeof(A[0])`

```
int n = sizeof(A) / sizeof(A[0]);
```

This finds the number of elements in the array.

Example:

```
int A[] = {8, 6, 3, 2, 5, 4};
```

There are 6 elements.

So:

```
n = 6
```

37. Why Selection Sort Uses Fewer Swaps

Selection Sort scans the unsorted part first.

It only swaps after it knows the smallest value.

Example:

```
8 6 3 2 5 4
```

During pass 1, it finds many smaller values:

- 6 is smaller than 8.
- 3 is smaller than 6.
- 2 is smaller than 3.

But it does not swap each time.

It only updates `k`.

Finally, it swaps once:

8 and 2

So each pass usually uses only one swap.

This is why Selection Sort is useful when we want to reduce the number of memory writes.

38. Selection Sort Time Complexity

Selection Sort uses nested loops.

The outer loop runs about $n - 1$ times.

The inner loop scans the remaining unsorted part.

For an array with n elements, the number of comparisons is roughly:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 1$$

This becomes:

$$n(n - 1) / 2$$

In Big O notation, this is:

$$O(n^2)$$

39. Best-Case Time Complexity

The best case happens when the array is already sorted.

Example:

2 3 4 5 6 8

A beginner may think:

The array is already sorted, so Selection Sort should stop early.

But Selection Sort does not know that immediately.

It still scans the remaining array for every position.

It still checks:

```
Is there anything smaller than 2?  
Is there anything smaller than 3?  
Is there anything smaller than 4?  
...
```

So even in the best case, Selection Sort still performs many comparisons.

Best-case time:

$O(n^2)$

40. Worst-Case Time Complexity

The worst case may happen when the array is reverse sorted.

Example:

8 6 5 4 3 2

Selection Sort must still scan the remaining array for each position.

It finds the smallest value and swaps it into position.

The number of comparisons is still about:

$(n - 1) + (n - 2) + \dots + 1$

Worst-case time:

$O(n^2)$

41. Why Best Case and Worst Case Are Both $O(n^2)$

Selection Sort does almost the same number of comparisons no matter what the input looks like.

If the array is sorted, it scans.

If the array is reverse sorted, it scans.

If the array is random, it scans.

So:

Case	Time complexity
Best case	$O(n^2)$
Average case	$O(n^2)$
Worst case	$O(n^2)$

This is why Selection Sort is called non-adaptive.

42. Space Complexity

Selection Sort sorts inside the original array.

It does not need another large array.

It only uses a few variables:

- `i`
- `j`
- `k`
- `temp` inside `swap()`

So space complexity is:

$O(1)$

This means constant extra space.

43. Is Selection Sort In-Place?

Yes.

Selection Sort is an in-place sorting algorithm.

That means it sorts mainly inside the original array.

It does not require a second array to store the sorted result.

So Selection Sort is:

In-place

44. Is Selection Sort Adaptive?

No.

Selection Sort is not adaptive.

Adaptive means:

The algorithm becomes faster when the data is already sorted or nearly sorted.

Selection Sort does not do this.

Even if the array is already sorted, it still scans the unsorted part.

Example:

2 3 4 5 6 8

Selection Sort still checks the remaining elements to confirm that no smaller value exists.

So Selection Sort is:

Non-adaptive

45. Selection Sort vs Bubble Sort Adaptiveness

Bubble Sort can become adaptive if we use a `flag`.

If no swap happens during a pass, Bubble Sort stops early.

Selection Sort does not have this same simple early-stopping behavior.

Algorithm	Adaptive?	Reason
Bubble Sort	Yes, if flag is used	Stops if no swap happens
Insertion Sort	Yes, naturally	Shifting stops early
Selection Sort	No	Still scans to find minimum

46. Is Selection Sort Stable?

Usually, no.

Selection Sort is normally not stable.

Stable means:

Equal values keep their original relative order.

Selection Sort can break this order because it uses long-distance swaps.

47. Stability Example

Suppose we have this array:

8A 3 5 8B 2

Here:

- `8A` and `8B` have the same value.
- `8A` came before `8B` originally.

Selection Sort finds the smallest value, which is `2`.

Then it swaps `2` with the first element, `8A`.

Result:

2 3 5 8B 8A

Now 8B comes before 8A .

The original order of equal values changed.

So Selection Sort is not stable.

48. Why Long-Distance Swaps Can Break Stability

In Bubble Sort, swaps happen only between neighboring values.

In Insertion Sort, equal values are not shifted past each other if we use `>` .

But in Selection Sort, one value can be swapped from far away.

That long-distance swap may move an earlier duplicate behind a later duplicate.

That is why Selection Sort is normally not stable.

49. Number of Swaps in Selection Sort

Selection Sort performs at most one swap per pass.

If there are `n` elements, there are at most `n - 1` passes.

So Selection Sort performs at most:

`n - 1` swaps

In Big O notation:

`O(n)` swaps

This is better than Bubble Sort in terms of swaps, because Bubble Sort may do many swaps.

50. Number of Comparisons in Selection Sort

Selection Sort still performs many comparisons.

For n elements, comparisons are approximately:

$$(n - 1) + (n - 2) + \dots + 1$$

This is:

$$n(n - 1) / 2$$

So comparisons are:

$$O(n^2)$$

Important idea:

Selection Sort reduces swaps, but it does not reduce comparisons.

51. Selection Sort Properties

Property	Selection Sort result
Sorting type	Comparison-based sorting
Main idea	Select the smallest value
Movement type	Swapping
Best-case time	$O(n^2)$
Average-case time	$O(n^2)$
Worst-case time	$O(n^2)$
Space complexity	$O(1)$
In-place?	Yes
Adaptive?	No
Stable?	No, normally
Number of swaps	At most $n - 1$

Property	Selection Sort result
Good for reducing swaps?	Yes
Good for already sorted data?	No special advantage
Good for huge data?	Usually no
Good for learning?	Yes

52. When Selection Sort Is Useful

Selection Sort is useful:

- For learning sorting.
- For understanding minimum selection.
- For understanding how indexes `i`, `j`, and `k` work.
- When you want fewer swaps.
- When memory writes are expensive.
- When you only need the smallest few elements.
- For small arrays.

Selection Sort is not usually used for very large datasets because its time complexity is $O(n^2)$.

53. K-Pass Usefulness

Selection Sort has a useful property:

After `k` passes, the `k` smallest values are fixed at the front.

Example:

Original:

8 6 3 2 5 4

After 1 pass:

2 6 3 8 5 4

Smallest value fixed:

2

After 2 passes:

2 3 6 8 5 4

Two smallest values fixed:

2 3

After 3 passes:

2 3 4 8 5 6

Three smallest values fixed:

2 3 4

This means Selection Sort can be useful if you only need the smallest few elements.

Example:

If you only need the smallest 3 values, you can run only 3 passes.

54. Bubble Sort K-Pass vs Selection Sort K-Pass

Algorithm	After k passes
Bubble Sort	k largest values are fixed at the end
Selection Sort	k smallest values are fixed at the front

Example:

Original:

8 6 3 2 5 4

After 3 Selection Sort passes:

2 3 4 | 8 5 6

The three smallest values are fixed at the front.

55. Selection Sort vs Bubble Sort

Feature	Bubble Sort	Selection Sort
Main idea	Push large values to the end	Select smallest value for the front
Movement	Many neighboring swaps	One swap per pass
Pass result	One largest value fixed at end	One smallest value fixed at front
Adaptive?	Yes, with flag	No
Stable?	Yes, if using <code>></code>	No, normally
Best case	<code>O(n)</code> with flag	<code>O(n²)</code>
Worst case	<code>O(n²)</code>	<code>O(n²)</code>
Extra memory	<code>O(1)</code>	<code>O(1)</code>
Good for few largest?	Yes	Not directly
Good for few smallest?	Not directly	Yes

56. Selection Sort vs Insertion Sort

Feature	Insertion Sort	Selection Sort
Main idea	Insert each value into sorted left part	Select smallest value for current position
Movement	Shifting	Swapping
Adaptive?	Yes, naturally	No
Stable?	Yes, if using <code>></code>	No, normally
Best case	<code>O(n)</code>	<code>O(n²)</code>
Worst case	<code>O(n²)</code>	<code>O(n²)</code>
Extra memory	<code>O(1)</code>	<code>O(1)</code>
Good for nearly sorted data?	Yes	No special advantage

Feature	Insertion Sort	Selection Sort
Swaps/writes	May shift many values	Few swaps

57. Which Is Better: Bubble, Insertion, or Selection?

There is no single best algorithm for every situation.

Each one teaches a different idea.

Bubble Sort is useful when:

- You are learning adjacent comparisons.
- You want to understand swapping.
- You want the largest values fixed at the end.
- You use the adaptive flag for nearly sorted data.

Insertion Sort is useful when:

- The array is already nearly sorted.
- You want a naturally adaptive algorithm.
- You want a stable algorithm.
- You are working with linked lists.

Selection Sort is useful when:

- You want fewer swaps.
- You want to find the smallest values one by one.
- You only need the smallest few elements.
- You want a simple algorithm that selects minimum values.

58. Common Beginner Mistakes

Mistake 1: Swapping Inside the Inner Loop

Wrong idea:

```
if (A[j] < A[k]) {
    k = j;
    swap(&A[i], &A[k]); // wrong place
}
```

This is wrong because Selection Sort should not swap immediately when it finds a smaller value.

Correct idea:

```
if (A[j] < A[k]) {  
    k = j;  
}
```

Then swap after the inner loop finishes:

```
swap(&A[i], &A[k]);
```

Mistake 2: Forgetting to Reset `k = i`

Wrong:

```
for (i = 0; i < n - 1; i++) {  
    for (j = i; j < n; j++) {  
        if (A[j] < A[k]) {  
            k = j;  
        }  
    }  
    swap(&A[i], &A[k]);  
}
```

Correct:

```
for (i = 0; i < n - 1; i++) {  
    k = i;  
    for (j = i; j < n; j++) {  
        if (A[j] < A[k]) {  
            k = j;  
        }  
    }  
    swap(&A[i], &A[k]);  
}
```

You must reset `k` at the beginning of every pass.

Mistake 3: Thinking Selection Sort Is Adaptive

Selection Sort is not adaptive.

Even if the array is already sorted, it still scans the remaining values.

So best-case time is still:

$O(n^2)$

Mistake 4: Thinking Fewer Swaps Means Stable

Selection Sort uses fewer swaps, but it is still usually not stable.

Why?

Because its swaps can be long-distance swaps.

Long-distance swaps can change the order of duplicate values.

Mistake 5: Confusing `j` and `k`

`j` is the scanner.

`k` stores the index of the smallest value found so far.

Do not swap with `j` at the end.

You should swap with `k`:

```
swap(&A[i], &A[k]);
```

Mistake 6: Thinking Every Pass Fully Sorts the Remaining Part

Selection Sort does not fully sort the remaining unsorted part in one pass.

Each pass only fixes one value.

After pass 1:

```
2 | 6 3 8 5 4
```

Only `2` is fixed.

The rest is still unsorted.

Mistake 7: Forgetting That the Last Element Does Not Need a Separate Pass

The outer loop should run until `n - 1`:

```
for (i = 0; i < n - 1; i++)
```

If all previous elements are placed correctly, the last element is automatically correct.

59. Key Vocabulary for Phase 4

Term	Simple meaning
Selection Sort	Sorting by repeatedly selecting the smallest remaining value
Minimum	The smallest value in a group
Sorted part	The part already fixed at the front
Unsorted part	The part still waiting to be sorted
<code>i</code>	Current position to fill
<code>j</code>	Scanner that searches the unsorted part
<code>k</code>	Index of the smallest value found so far
Swap	Exchange two values
Pass	One full process of selecting the minimum for a position
Non-adaptive	Does not become faster on already sorted data
Not stable	May change the order of equal values
In-place	Sorts inside the original array
$O(n^2)$	Quadratic time
$O(1)$ space	Constant extra memory

60. Selection Sort Evaluation Checklist

When studying Selection Sort, ask these questions.

Comparisons

Does it compare values?

Yes.

It compares:

$A[j] < A[k]$

It uses this comparison to find the smallest value.

Movement

Does it move values?

Yes.

It moves values using swaps.

But it performs fewer swaps than Bubble Sort.

Adaptiveness

Is it adaptive?

No.

It still scans the remaining array even if the array is already sorted.

Memory

Does it need a large extra array?

No.

It only uses a few variables.

Space complexity:

$O(1)$

Stability

Is it stable?

Usually, no.

Long-distance swaps can change the order of equal values.

Best Case

What is the best case?

Already sorted data.

But Selection Sort still scans.

Best-case time:

$O(n^2)$

Worst Case

What is the worst case?

Reverse sorted or random data can still require full scanning.

Worst-case time:

$O(n^2)$

K-Pass Result

What happens after passes?

The `k` smallest values are fixed at the front.

61. Complete Phase 4 Program With Comments

```
#include <stdio.h>

// Function to swap two integer values using pointers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Selection Sort function
void SelectionSort(int A[], int n) {
    int i, j, k;

    // Outer loop chooses the position to fill
    for (i = 0; i < n - 1; i++) {
        k = i; // assume the current position has the smallest value

        // Inner loop searches the unsorted part
        for (j = i; j < n; j++) {
            if (A[j] < A[k]) {
                k = j; // update index of smallest value
            }
        }

        // Swap the smallest value with the current position
        swap(&A[i], &A[k]);
    }
}

int main() {
    int A[] = {8, 6, 3, 2, 5, 4};
    int n = sizeof(A) / sizeof(A[0]);

    SelectionSort(A, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
}
```

```
    return 0;
}
```

Expected output:

Sorted array: 2 3 4 5 6 8

62. Phase 4 Summary

Phase 4 teaches Selection Sort.

The main idea is:

Select the smallest value from the unsorted part and place it at the front.

Important points:

- Selection Sort is a comparison-based sorting algorithm.
- It works from left to right.
- It divides the array into a sorted part and an unsorted part.
- The sorted part grows at the front.
- It uses `i`, `j`, and `k`.
- `i` is the current position.
- `j` scans the unsorted part.
- `k` stores the index of the smallest value found so far.
- It swaps only after the inner loop finishes.
- After each pass, one smallest value is fixed at the front.
- After `k` passes, the `k` smallest values are fixed at the front.
- It uses fewer swaps than Bubble Sort.
- It performs at most `n - 1` swaps.
- It still performs $O(n^2)$ comparisons.
- Best-case time is $O(n^2)$.
- Average-case time is $O(n^2)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- It is in-place.
- It is not adaptive.
- It is normally not stable.
- It is useful when reducing swaps is important.

63. One-Sentence Memory Trick

Remember Selection Sort like this:

Select the smallest remaining value and swap it into the front.

Even shorter:

Select minimum, swap once.

64. What Comes Next

After Phase 4, you have completed the three basic simple comparison sorts:

Phase	Algorithm / Topic	Main idea
Phase 1	Foundations	Learn sorting vocabulary and evaluation criteria
Phase 2	Bubble Sort	Push large values to the end using adjacent swaps
Phase 3	Insertion Sort	Insert each value into the sorted left part
Phase 4	Selection Sort	Select the smallest value and place it at the front

Next, Phase 5 introduces Quick Sort partitioning.

Quick Sort is more advanced because it uses a pivot and partitioning.

Selection Sort prepares you for this idea because both Selection Sort and Quick Sort think about placing values into useful positions.

But Selection Sort places the smallest value at the front, while Quick Sort places a pivot into its correct sorted position.

65. Final Beginner Takeaway

Selection Sort is simple but not very fast for large data.

It is important because it teaches how to select the minimum value and place it in the correct position.

The most important thing to remember is:

Selection Sort chooses a position, finds the smallest remaining value, and swaps it into that position.

If you understand `i`, `j`, and `k`, you understand the heart of Selection Sort.